Alexandria University
Faculty of Engineering
Computer and Systems Department

CSE: NLP
Assigned: Th, Oct 9, 2025
Due: Su, Oct 19, 2025

Lab # 1

Part 1: N-Gram Language Model

Objective

An N-gram language model is one of the earliest and simplest approaches to language modeling. In this section, you will construct an N-gram model step-by-step to gain a better understanding of probabilistic text modeling.

Dataset

Dataset: `Gutenberg Corpus` You can access it from the `nltk` library in Python.
You should use:

- **Training Corpus:** `shakespeare-caesar` and `shakespeare-macbeth`.
- **Evaluation Corpus:** `shakespeare-hamlet`.

Steps and Requirements

Data Preprocessing

- Tokenize the text into words.
- Apply text normalization steps such as:
  - Converting all text to lowercase.
  - Removing punctuation and special characters.
  - Optionally normalization and stopwords removal (depending on experimental design).

Constructing the N-Gram Model

- Begin with a **unigram** model as a baseline.
- Extend the model to **bigrams**, **trigrams**, or a chosen N-gram size.
- Implement **Laplace smoothing** to handle unseen N-grams during testing.
- Implement the **generation** function of you model.

Evaluation: Perplexity

- Train your N-gram model on the **training corpus**.
- Compute the **perplexity** on the **evaluation corpus** to evaluate the model's performance.
- Compare the performance across different N-gram sizes (up to n=10).

Expected Outcome

By the end of this task, you should:

- Understand how to build and evaluate a statistical language model.
- Be able to compare model performance across different N-gram configurations.
- Recognize the role of smoothing in improving generalization for unseen sequences.

Part 2: Text Classification

Dataset

Dataset: `avsolatorio/mteb-emotion-avs_triplets`
For this assignment, you will use the **Emotion Recognition** dataset that include 6 emotion classes. This dataset consists of labeled sentences with he goal of predicting one of 6 emotion classes: `anger`, `sadness`, `surprise`, `love`, `joy`, and `fear`. The dataset is accessible on `Huggingface` through the following link.

Preprocessing for Classification

Preprocess each sentence with pre-processing pipeline similar to **Part 1**.

Part 2.1: Naïve Bayes Classification

Algorithm Implementation

Implement a **Naïve Bayes classifier** from scratch using only `NumPy` (no external libraries) by following the procedures below:

- Compute the **prior probabilities** for each class.
- Estimate the **likelihood** of each token given a class.
- For unseen text, infer the label by **maximizing the posterior probability**.

Comparison with `scikit-learn`

After implementing your model, reproduce the same classification task using the `scikit-learn` library.

- Use `CountVectorizer` to transform text into a **bag-of-words** representation.
- Use `MultinomialNB` for classification.
- Evaluate both models using the metrics implemented in **Part 2.3**.

Part 2.2: Logistic Regression

Feature Representation

Convert the text after preprocessing to **bi-grams** and construct the feature vector for each sentence based on the bi-grams' existence:

1. Build a vocabulary of all unique bi-grams in the dataset.
2. Each sentence should be represented by a sparse binary vector, where each feature corresponds to the presence (1) or absence (0) of a specific bi-gram.

Discussion. Is this a good way of representing features? Briefly discuss the advantages and limitations (no code required).

Algorithm Implementation

Implement **logistic regression** from scratch using only `NumPy`, following these steps:

- Initialize model parameters (weights and bias).
- Choose an appropriate learning rate and train using (mini-)batch or stochastic gradient descent.
- Update parameters iteratively to minimize the **cross-entropy loss**:
  1. Compute scores for the batch/sample.
  2. Compute class probabilities and loss.
  3. Compute gradients.
  4. Apply gradient updates using the chosen learning rate.
- Shuffle the dataset each epoch.

Comparison with `scikit-learn`

Compare your implementation against two built-in classifiers:

- `sklearn.linear_model.LogisticRegression`
- `sklearn.linear_model.SGDClassifier`

Discuss the following points:

- Which approach performs better, and under what conditions?
- Which implementation is most similar to your version?

Part 2.3: Confusion Matrix & Evaluation Metrics

Implementation

Implement the following components using only `NumPy`:

1. A function to compute the **confusion matrix**, given predictions and true labels.
2. From the confusion matrix, compute the following (per class and macro-averaged):
   - Precision
   - Recall
   - F1-score

Comparison with `scikit-learn`

Compare your from-scratch results against the following functions:

- `sklearn.metrics.confusion_matrix`
- `sklearn.metrics.precision_score`
- `sklearn.metrics.recall_score`
- `sklearn.metrics.f1_score`

Expected Outcome

By the end of this part, you should:

- Understand how to preprocess labeled sentences for text classification.
- Implement Naïve Bayes and Logistic Regression classifiers from scratch.
- Compare the performance of your models with standard `scikit-learn` implementations and understand differences between them and your implementation.
- Derive evaluation metrics manually and validate them against reference results and understand the meaning of macro-averaging.